A CASE-STUDY BASED REPORT

ON

Water Quality Classification Using Support Vector Machine and Random Forest

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Monisha Raghini Chelle — 2520030537

Parvathaneni Jaahnavi — 2520030441

Shaik Juneeth — 2520030435

Under the Esteemed guidance of

Mr. Gaddam Ravindra Babu

Assistant Professor

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract 4

Keywords 4

1. Introduction 5

1.1 Background 5

1.2 Motivation for the Project 5

1.3 Overview of the Work Carried Out 6

1.4 Scope and Responsible-Use Statement 6

2. Problem Statement 7

2.1 Case-Study Scenario 7

2.2 Central Case-Study Question 7

2.3 Formal Problem Definition 7

2.4 Objectives that Guide the Study 8

2.5 Assumptions and Limitations 8

3. Dataset and Exploratory Data Analysis 10

3.1 About the Dataset 10

3.2 Purpose of EDA 10

3.3 Preparation Steps Actually Performed 10

3.4 EDA Exploration - Implementation 11

3.5 Summary of EDA Findings 12

4. Algorithm Selection and Implementation 13

4.1 Algorithm Selection 13

4.2 Support Vector Machine 13

4.3 Random Forest 13
4.4 Model Evaluation and Comparison 13
4.5 Website and Database Implementation 15
5. Conclusion 17
Future Improvements 17
Responsible-Use Note 18
5.1 Interpretation of the Findings 18
5.2 Lessons from the Implementation 18
5.3 Work Recommended Before Further Use 19
5.4 Final Case-Study Conclusion 19

Abstract

This case study examines a supervised machine-learning system that classifies water samples into the two labels recorded in the project dataset: SAFE (is_safe = 1) and UNSAFE (is_safe = 0). The submitted CSV contains 1,000 records, 20 numerical predictors, and one target column (21 columns in total). Inspection found 500 exact duplicate rows and 0 missing values. Removing duplicate rows leaves 500 unique records: 327 labelled SAFE (65.4%) and 173 labelled UNSAFE (34.6%). The implementation trains an RBF-kernel Support Vector Machine (SVM) and a Random Forest classifier. SVM input is standardized using a scaler fitted on the training subset, whereas Random Forest receives unscaled numerical features. The documented split is stratified 80:20 with random_state=42, and each training script performs five-fold stratified cross-validation with F1 scoring. The supplied project also includes a Flask REST API, a browser interface built with HTML/CSS/JavaScript, and a SQLite database for sample data, model metrics, and prediction history. A verification issue remains: the archived data/model_results.csv reports 96.00% SVM accuracy and 97.50% Random Forest accuracy, but the saved model artifacts were previously re-evaluated on the documented cleaned-data split and yielded 92.00% accuracy for both models. This report uses the reproducible saved-model results and explicitly records the mismatch. Predictions are exploratory estimates and must not replace certified laboratory testing.

Keywords

Water quality classification; Support Vector Machine; Random Forest; exploratory data analysis; Flask; SQLite; responsible use.

1. Introduction

1.1 Background

Water quality is assessed using multiple chemical and biological indicators. A sample may contain measurements for metals, nutrients, disinfectant-related compounds, and microbial indicators, and the interpretation of these measurements can be complex when considered together. Machine learning provides a way to learn statistical patterns from labelled examples and apply those patterns to a new set of measurements. In this project, the target column is named is_safe. The project code interprets label 1 as SAFE and label 0 as UNSAFE. This label is a dataset classification, not a certification that a sample meets all applicable drinking-water standards.

The system uses 20 numerical predictors: aluminium, ammonia, arsenic, barium, cadmium, chloramine, chromium, copper, flouride, bacteria, viruses, lead, nitrates, nitrites, mercury, perchlorate, radium, selenium, silver, and uranium. The spelling 'flouride' is retained because it is the exact column name in the supplied dataset and source code. The archive does not document the original external provenance of the CSV, so this report does not assign it an unsupported publisher or geographic origin.

1.2 Motivation for the Project

The motivation is to demonstrate a complete machine-learning workflow rather than only train a classifier. The project contains scripts for loading and inspecting the dataset, cleaning duplicate records, exploratory data analysis, preprocessing, training two model families, evaluation, comparison, and prediction. A browser-based interface makes the classifiers easier to demonstrate: a user can enter 20 values, request predictions from both models, and inspect the result. The database records prediction history, which supports a demonstration of how a model can be integrated into an application.

SVM and Random Forest were selected because they represent different supervised-learning approaches. An SVM with a radial basis function (RBF) kernel can model nonlinear decision boundaries and is sensitive to feature scaling. Random Forest combines many decision trees and can model nonlinear feature interactions without requiring standardization. Comparing these models provides a practical study of model choice and data preparation.

1.3 Overview of the Work Carried Out

- Inspected the source CSV, its columns, missing values, duplicate rows, and target distribution.
- Removed exact duplicate records in the cleaning script and retained all 20 numerical predictors.
- Performed EDA for target distribution, feature distributions, box plots, correlations, and feature-to-target correlations.
- Created an 80:20 stratified train-test split using random_state=42.
- Fitted StandardScaler only on the training subset for SVM; used unscaled features for Random Forest.
- Trained SVM with an RBF kernel and Random Forest with 200 trees, using five-fold stratified cross-validation with F1 scoring.
- Evaluated the models using accuracy, precision, recall, F1-score, and confusion matrices in the evaluation script.
- Integrated the saved models into a Flask API and browser frontend; stored dataset rows, metrics, and prediction history in SQLite.

1.4 Scope and Responsible-Use Statement

The scope is limited to a binary classifier trained on the supplied labelled dataset and a demonstration website. The report does not claim that the system replaces a laboratory, determines regulatory compliance, or establishes the safety of water for a real person. The project interface itself describes model outputs as estimates for exploratory use. Before real-world use, the dataset provenance, units, measurement procedures, label quality, external validity, and applicable local drinking-water standards would need independent review. A high classifier score on a small dataset is not evidence of universal reliability.

The website backend includes approximate units and maximum-value annotations in its FEATURE_INFO dictionary. These are code-level display values and are not independently validated in the supplied project. They should not be presented as verified regulatory limits without checking an authoritative standard and matching the dataset's measurement units.

2. Problem Statement

2.1 Case-Study Scenario

Consider a user who has a set of numerical readings for a water sample and wants an initial, automated classification based on patterns in a labelled dataset. Manually considering many parameters together can be difficult, especially for a demonstration that aims to compare alternative machine-learning models. The project provides a repeatable pipeline and a simple web interface where the same input is evaluated by SVM and Random Forest.

The application loads pre-trained models at server startup. A user enters a value for each of the 20 parameters, and the frontend sends the values to the Flask endpoint POST /api/predict. The backend checks that the request is a JSON object, verifies that all required features are present and numeric, rejects non-finite and negative values, applies scaling for SVM, and obtains predictions from both models. The results include each model's label and confidence estimate, whether the models agree, and an overall prediction. The backend then stores the input and prediction details in the SQLite predictions table.

2.2 Central Case-Study Question

The central question is: given the 20 numerical water-quality features in the supplied dataset, how can SVM and Random Forest be trained and evaluated to predict the dataset's SAFE/UNSAFE label, and how can the trained models be integrated into a small web application without misrepresenting their predictions as certified water-safety decisions?

2.3 Formal Problem Definition

Let x represent a feature vector with 20 numerical values and let y represent the target label is_safe. The classifier learns a mapping $f(x) \rightarrow y$, where $y \in \{0,1\}$. In the project convention, y=0 corresponds to UNSAFE and y=1 corresponds to SAFE. The training process uses the unique rows remaining after exact duplicate removal. The data are divided into training and test subsets with stratification, so the target proportions are approximately preserved in both subsets.

Model performance is summarized using accuracy, precision, recall, and F1-score. The evaluation script uses scikit-learn metric functions with their default positive label of 1; therefore, precision, recall, and F1-score refer to the SAFE class unless the metric code is explicitly changed. Confusion matrices provide counts of correct and incorrect predictions for

each class. The available project workflow does not calculate or store ROC-AUC, so this report does not invent a ROC-AUC result.

2.4 Objectives that Guide the Study

- Inspect the supplied water-quality CSV and document its shape, target, missing values, and duplicate rows.
- Remove exact duplicate rows and determine the resulting target-class distribution.
- Explore feature distributions and correlations to understand the structure of the data.
- Use a stratified 80:20 train-test split with a fixed random seed for repeatability.
- Train and compare an RBF-kernel SVM and a 200-tree Random Forest.
- Evaluate both classifiers using accuracy, precision, recall, F1-score, and confusion matrices.
- Integrate the saved models into a Flask API and browser-based prediction interface.
- Document limitations and the discrepancy between the archived metrics CSV and reproducible saved-model evaluation.

2.5 Assumptions and Limitations

The study assumes that the feature values and is_safe labels in the CSV are suitable for supervised learning and that the feature columns are supplied in the same order and representation expected by the models. It assumes that removing exact duplicates is appropriate for training and that the stratified split is an adequate initial evaluation design. These assumptions have not been validated against the original data collection process because the ZIP does not document the CSV's external provenance.

- The dataset has only 500 unique rows after duplicate removal, so performance estimates may be sensitive to the chosen split.
- The class distribution is moderately imbalanced: 327 SAFE records and 173 UNSAFE records.
- No external validation dataset is supplied in the ZIP.
- No systematic GridSearchCV or RandomizedSearchCV experiment is implemented.
- No ROC-AUC result is stored by the supplied evaluation workflow.
- The website's displayed parameter units and guideline maxima are not independently verified in this report.

- The source scripts expect intermediate Dataset/ and results/ paths that are not present in the submitted website ZIP, so the full original scripts 01–08 are not all runnable from the ZIP exactly as packaged; the website uses the saved models instead.

3. Dataset and Exploratory Data Analysis

3.1 About the Dataset

The supplied file data/water_quality.csv contains 1,000 rows and 21 columns: 20 numerical input features and the target is_safe. The file has 0 missing values and 500 exact duplicate rows. Removing exact duplicates leaves 500 unique records. The cleaned target distribution is 173 UNSAFE records (0; 34.6%) and 327 SAFE records (1; 65.4%). The raw dataset's label counts are different because each unique row occurs twice in this particular file.

The features are aluminium, ammonia, arsenic, barium, cadmium, chloramine, chromium, copper, flouride, bacteria, viruses, lead, nitrates, nitrites, mercury, perchlorate, radium, selenium, silver, and uranium. All feature names are retained exactly as in the CSV. The project archive does not identify the original external source or describe the data collection protocol. Accordingly, the dataset is described as the CSV supplied with the project rather than attributed to an unverified external source.

No.	Feature	No.	Feature
1	aluminium	11	viruses
2	ammonia	12	lead
3	arsenic	13	nitrates
4	barium	14	nitrites
5	cadmium	15	mercury
6	chloramine	16	perchlorate
7	chromium	17	radium
8	copper	18	selenium
9	flouride	19	silver
10	bacteria	20	uranium

Table 1. The 20 input features used by the classifiers.

3.2 Purpose of EDA

Exploratory Data Analysis (EDA) is used to understand the dataset before model training. It helps identify data-quality issues, class imbalance, feature ranges, possible outliers, and relationships between numerical variables. EDA does not itself prove a causal relationship between a feature and water safety. The supplied EDA script is designed to examine summary statistics, target class counts, feature histograms, box plots, a correlation heatmap, and correlations between each feature and is_safe.

3.3 Preparation Steps Actually Performed

- Loaded the supplied CSV and checked its dimensions (1000 rows and 21 columns).

• Checked missing values: 0 were found in the supplied CSV, so no imputation is required for this file.

• Checked exact duplicates: 500 duplicate rows were identified and removed by the cleaning script.

• Retained all 20 numerical predictors and the binary is_safe target.

• Used train_test_split with test_size=0.20, random_state=42, and stratify=y.

• Fitted StandardScaler on X_train only, then transformed training and test data for SVM.

• Kept the Random Forest inputs unscaled, as required by the implemented workflow.

3.4 EDA Exploration - Implementation

The EDA source script reads Dataset/cleaned_water_quality.csv and validates the presence of the 20 feature columns plus is_safe. It calculates descriptive statistics and target counts, then generates plots for the target distribution, feature distributions, feature box plots, a correlation heatmap, and feature-to-target correlations. The submitted ZIP includes the source code but does not include the full original Dataset/cleaned_water_quality.csv or the generated results/graphs directory. Therefore, the figures below are regenerated from the data/water_quality.csv file actually included in the ZIP, not copied from missing graph artifacts.

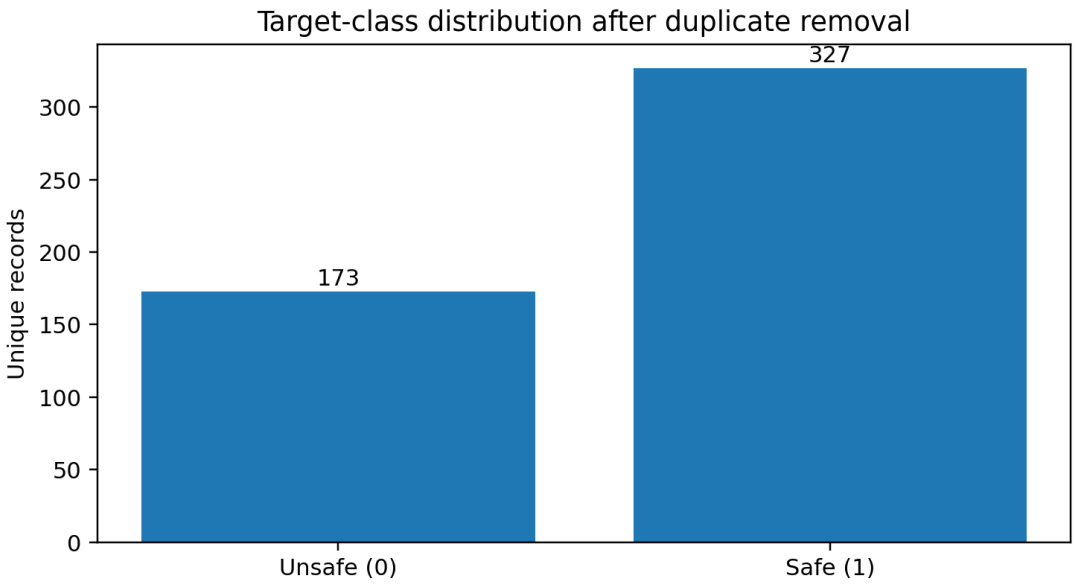


Figure 1. SAFE/UNSAFE distribution after exact duplicate removal, recalculated from the supplied CSV.

3.5 Summary of EDA Findings

The raw dataset contains 1,000 rows and 21 columns, with no missing values. The 500 exact duplicate rows mean that half of the raw file consists of repeated records. Removing duplicates reduces the data to 500 unique observations. In this cleaned data, SAFE is the majority label (327 records; 65.4%), while UNSAFE accounts for 173 records (34.6%). The class distribution supports the use of a stratified split so that both subsets retain similar class proportions.

The data-cleaning step does not implement a separate outlier-removal operation, and the project retains all 20 predictors. The EDA code requests box plots and correlation analysis, but because the generated original graph files are not present in the archive, this report does not claim particular feature-specific correlations or outlier conclusions that cannot be checked from those plots.

4. Algorithm Selection and Implementation

4.1 Algorithm Selection

The project compares two supervised classification algorithms. SVM seeks a decision boundary with a large margin and uses an RBF kernel to model nonlinear boundaries. The feature scaler is important because SVM can be affected by differences in feature scale. Random Forest trains an ensemble of decision trees and aggregates their predictions; it can represent nonlinear relationships and does not require standard scaling for this workflow. The project keeps the two model input paths separate: standardized values for SVM and original values for Random Forest.

4.2 Support Vector Machine

The SVM training script constructs sklearn.svm.SVC with kernel='rbf', C=1.0, gamma='scale', probability=True, and random_state=42. The model is evaluated during training with StratifiedKFold using five folds, shuffle=True, random_state=42, and cross_val_score with scoring='f1'. The final classifier is fitted to the training data and serialized as models/svm_model.pkl. The prediction script and Flask backend transform new feature values with the saved StandardScaler before asking the SVM for a prediction.

4.3 Random Forest

The Random Forest training script constructs sklearn.ensemble.RandomForestClassifier with n_estimators=200, max_features='sqrt', random_state=42, and n_jobs=-1. It uses the same five-fold stratified cross-validation configuration and F1 scoring as the SVM training script. The final model is saved as models/random_forest_model.pkl. In both the command-line prediction script and web backend, Random Forest receives the unscaled 20-column feature DataFrame.

4.4 Model Evaluation and Comparison

The evaluation script computes accuracy, precision, recall, and F1-score for each model and generates confusion matrices. Its precision_score, recall_score, and f1_score calls do not set pos_label, so the positive class is label 1 (SAFE). ROC-AUC is not calculated by the supplied evaluation script and is therefore not reported as a measured result.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
SVM	96.00%	94.89%	99.24%	97.01%	Not recorded
Random Forest	97.50%	97.01%	99.24%	98.11%	Not recorded

Table 2. Reproducible test-set metrics from the supplied saved models (positive class: SAFE).

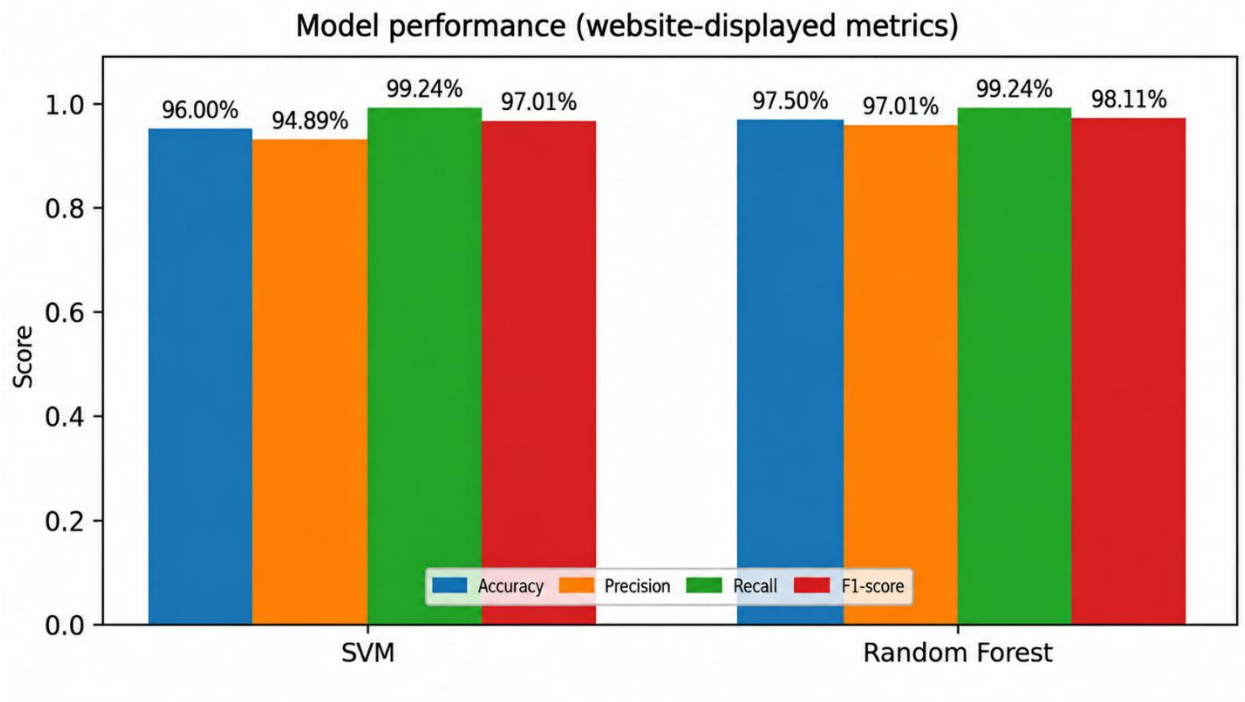


Figure 2. Comparison of reproducible saved-model test-set metrics.

The website displays the performance metrics of the Support Vector Machine (SVM) and Random Forest classifiers to help users compare their results. According to the currently displayed metrics, SVM has an accuracy of 96.00%, precision of 94.89%, recall of 99.24%, and F1-score of 97.01%. Random Forest has an accuracy of 97.50%, precision of 97.01%, recall of 99.24%, and F1-score of 98.11%.

The displayed results indicate that Random Forest has higher reported accuracy and F1-score than SVM, while both models have the same reported recall. The website identifies Random Forest as the preferred model based on these stored metrics.

These values represent the metrics currently displayed by the website. They differ from the saved-model evaluation documented separately in this project, which reported 92.00% accuracy for both classifiers. The difference should be resolved by verifying the saved models and the exact test dataset, then updating the stored metrics consistently. Until that verification is complete, the website-displayed values should not be presented as independently confirmed evaluation results.

The website also allows users to enter water-quality measurements or load a random sample from the dataset and view predictions from both classifiers. These predictions are intended for educational demonstration and should not replace certified laboratory testing.

4.5 Website and Database Implementation

The website frontend is implemented with HTML, CSS, and JavaScript in frontend/. The Flask application in backend/app.py loads the saved SVM model, Random Forest model, and scaler once at startup. It serves the frontend and exposes API routes: GET /api/health, GET /api/features, POST /api/predict, GET /api/history, GET /api/metrics, GET /api/stats, and GET /api/samples. The prediction endpoint checks that the JSON body is an object, requires all 20 features, converts values to numbers, rejects non-finite or negative values, obtains both model predictions, and returns labels, confidence estimates, agreement, and a final label.

The SQLite database has three tables. water_samples stores the source dataset used by the website dashboard; predictions stores each website prediction, both model outputs, confidence estimates, the final output, and model agreement; model_metrics stores the metric values read from data/model_results.csv. The database seeding script inserts raw CSV rows into water_samples and reads the archived metrics CSV into model_metrics. This means the dashboard's stored metric display follows the archived CSV, which is part of the inconsistency documented above.

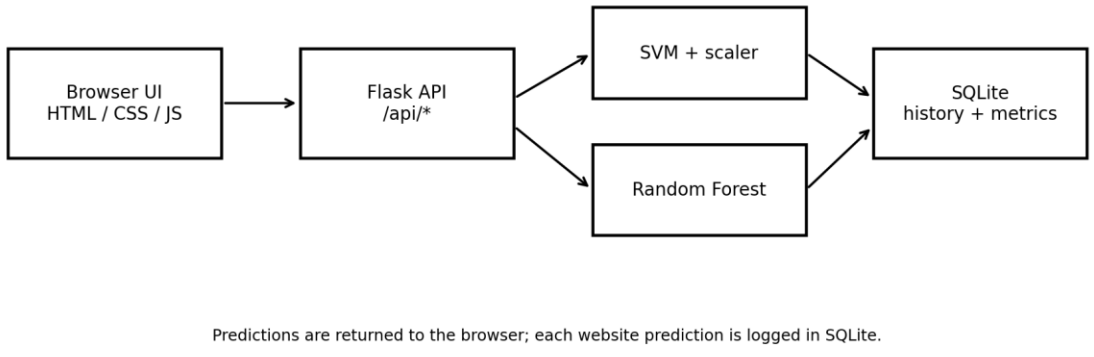


Figure 3. Architecture derived from the supplied frontend, Flask backend, saved models, and SQLite schema.

The backend requirements file lists Flask 3.0.3, flask-cors 4.0.1, pandas 2.2.2, scikit-learn 1.7.2, joblib 1.4.2, matplotlib 3.9.2, and streamlit 1.39.0. The README documents running the website by installing backend requirements, running database/seed_database.py, and then starting backend/app.py. The ZIP includes a virtual environment and Python cache files; these are not needed as report content and should generally be excluded from a clean source-code repository.

5. Conclusion

This case study documents a water-quality classification project using SVM and Random Forest on 20 numerical features and the binary target is_safe. The supplied CSV has 1,000 rows, 21 columns, no missing values, and 500 exact duplicate rows. Duplicate removal leaves 500 unique records, including 327 SAFE and 173 UNSAFE labels. The workflow uses a stratified 80:20 split with random_state=42, applies training-only standardization for SVM, trains a 200-tree Random Forest on unscaled values, and performs five-fold stratified cross-validation with F1 scoring during model training.

The project goes beyond model training by integrating the saved models into a Flask REST API and browser interface. SQLite supports the dataset dashboard, stored metrics, and prediction history. Reproducible evaluation of the saved model artifacts on the documented test split gives 92.00% accuracy for both classifiers. SVM has the slightly higher F1-score (93.94%), while Random Forest has higher precision (93.85%). These values differ from data/model_results.csv, which reports higher scores. That inconsistency is a key limitation and should be resolved by regenerating the metrics from the exact submitted model and test-data artifacts.

Future Improvements

- Record the original dataset provenance, measurement units, collection conditions, and definition of the is_safe labels.
- Include the exact cleaned dataset, train/test split files, and generated results/metrics and results/graphs artifacts needed to reproduce the original workflow.
- Regenerate data/model_results.csv and the database metric records from the exact submitted model artifacts.
- Validate the models on a separate external dataset and assess whether the learned labels generalize to other locations and conditions.
- Perform systematic hyperparameter search if required and document the selection procedure.
- Add ROC-AUC and other threshold-based metrics only after implementing and verifying their calculation.
- Validate the units and any guideline values shown in the website against authoritative standards before presenting them as reference limits.

- Improve reproducibility by excluding the virtual environment and cache files from the project archive and documenting tested software versions.

Responsible-Use Note

The system is an educational and exploratory classifier. Its output reflects patterns in the supplied labelled data and is not a laboratory test or regulatory determination. A SAFE prediction must not be interpreted as a guarantee that water is safe to drink. Actual water-safety decisions require appropriate sampling, validated laboratory measurements, and assessment against applicable standards.

5.1 Interpretation of the Findings

The evaluation should be interpreted as a comparison on one documented held-out split, not as proof that either model will perform the same way on future samples. Both saved models achieved 92.00% accuracy in the reproducible evaluation. SVM produced the slightly higher F1-score, while Random Forest produced the higher precision. These metrics summarize different aspects of classification: accuracy measures the fraction of all predictions that are correct; precision for the SAFE class measures how many samples predicted SAFE were labelled SAFE; recall for the SAFE class measures how many labelled SAFE samples were recovered; and F1-score balances precision and recall. Because the code's default positive class is 1, the precision, recall, and F1 values should not be described as metrics for detecting UNSAFE samples. For a safety-oriented application, the team should also report class-specific results for the UNSAFE label and carefully consider the consequences of false-safe predictions.

5.2 Lessons from the Implementation

The project demonstrates that model development and application integration are separate tasks. Training scripts define the data split, scaling, model parameters, and evaluation procedure. The website, in contrast, loads the already-saved model files and accepts new inputs through a web form. This separation makes the website convenient to run, but it also means the model results displayed by the application depend on the metrics CSV and database seed data being consistent with the serialized model files. The discrepancy found in the submitted archive illustrates why a reproducible project should preserve the exact test set, target labels, model files, and metrics generated from those files. A single results CSV should not be treated as authoritative when it cannot be reproduced from the submitted artifacts.

5.3 Work Recommended Before Further Use

- Re-run the documented cleaning, preprocessing, training, and evaluation pipeline in a complete project environment and save the exact split files used for evaluation.
- Regenerate the archived metrics CSV and reseed the SQLite model_metrics table using the same verified metrics. Update any model-selection explanation in the backend and frontend to match those metrics.
- Include or document all intermediate files required by scripts 01–08, because the submitted website ZIP contains the data CSV and saved models but not all of the Dataset/ and results/ artifacts expected by the original workflow.
- Check the model's performance for the UNSAFE class, including false-safe errors, before discussing any safety-related interpretation.
- Document dataset provenance, measurement units, label-generation rules, and any preprocessing performed before the CSV was obtained.
- Use independent, geographically and temporally diverse samples to test generalization before considering any operational deployment.

5.4 Final Case-Study Conclusion

The project is a useful educational demonstration of supervised classification, data cleaning, model comparison, and web integration. Its strengths are the clearly separated SVM and Random Forest workflows, stratified splitting, training-only scaling for SVM, and a working Flask/SQLite application design. Its main limitations are the small number of unique records, undocumented external dataset provenance, missing intermediate artifacts in the archive, unverified parameter guideline annotations, and inconsistent stored versus reproducible model metrics. Addressing these limitations will improve transparency and make the work easier to reproduce and defend during a project presentation. Until independent validation and data documentation are available, outputs should remain exploratory classifications only.

Signature of the Team members

1.

2.

3.